

Decisões de Implementação

Resolvedor de Sudoku — Paradigma Imperativo (C)

Decisões de Implementação

Este documento resume, de forma breve, as principais decisões de projeto tomadas na implementação em C do resolvedor de Sudoku, cobrindo estrutura de dados, algoritmo de resolução, tratamento de erros e organização do código.

1. Representação do tabuleiro

O tabuleiro é representado como uma matriz simples `int[9][9]`, com 0 indicando célula vazia. Optou-se por essa representação, em vez de uma estrutura mais elaborada (ex: lista de células), por ser a forma mais direta de acesso $O(1)$ a qualquer posição — algo usado repetidamente tanto na validação quanto na resolução.

2. Validação: listas encadeadas em vez de laços diretos

A validação (`ValTabuleiro`) foi implementada agrupando, para cada dígito de 1 a 9, todas as posições em que ele aparece (lista `NoD`), e comparando cada par de posições entre si (mesma linha, coluna ou bloco). Os conflitos encontrados são registrados numa segunda lista encadeada (`NoE`), permitindo reportar **todos** os conflitos do tabuleiro de uma vez, e não apenas o primeiro encontrado. A checagem de bloco usa a divisão inteira `linha/3` e `coluna/3`, que agrupa corretamente os índices 0–8 em três grupos de três.

3. Completude como checagem compartilhada

A função `estaCompleto` verifica se o tabuleiro não possui nenhuma célula vazia, e é reutilizada em dois pontos distintos, em vez de duplicar essa lógica:

- **Na validação (case 1):** distingue um tabuleiro sem conflitos mas ainda parcialmente vazio (“Tabuleiro valido!”) de um tabuleiro sem conflitos e totalmente preenchido (“Tabuleiro valido e completo!”).
- **Na resolução (case 2):** ao confirmar que o tabuleiro já chegou completo (e já validado como correto pela checagem anterior), o programa evita chamar `resolverSudoku` à toa e imprime uma mensagem específica informando que nenhuma busca foi necessária.

Centralizar essa lógica numa única função evita que os dois pontos de uso divirjam entre si com o tempo (por exemplo, um deles sendo corrigido e o outro não).

4. Resolução: backtracking com heurística MRV e bitmask

Para a resolução (`resolverSudoku`), optou-se por backtracking com duas otimizações:

- **Bitmask (EstadoBitmask):** os dígitos já usados em cada linha, coluna e bloco são representados como inteiros de 9 bits (um bit por dígito). Isso reduz a checagem de candidatos de três laços de 9 iterações para três operações bit a bit (`|`, `~`, `&`), tornando cada tentativa de preenchimento muito mais rápida.
- **Heurística MRV (Minimum Remaining Values):** em vez de preencher células em ordem fixa, o algoritmo sempre escolhe a célula vazia com **menos candidatos possíveis**. Isso reduz drasticamente o número de tentativas malsucedidas, já que células quase decididas eliminam ambiguidade cedo na busca.

Por que uma struct por ponteiro, e não variáveis globais: os três vetores de bitmask foram agrupados em `EstadoBitmask` e passados por ponteiro para todas as funções de resolução. Essa escolha deixa explícito, na assinatura de cada função, qual estado ela lê e/ou modifica — em vez de depender implicitamente de variáveis globais, o que dificultaria tanto a leitura do código quanto a reutilização das funções em outro contexto.

5. Validação prévia antes de resolver

No case 2 do menu, o tabuleiro é validado (`ValTabuleiro`) antes de se tentar resolvê-lo. Essa checagem evita

rodar o backtracking sobre um tabuleiro já inconsistente (com conflitos na entrada), o que geraria resultados sem sentido ou tempo de execução desperdiçado em um problema sem solução válida por definição. Em seguida, a mesma checagem de completude (`estaCompleto`) descrita na seção 3 decide se o backtracking chega a ser executado ou se o programa apenas confirma que o tabuleiro já está pronto.

6. Validação de resposta em três etapas

A validação de uma resposta fornecida pelo usuário (case 3) foi dividida em três checagens sequenciais, na ordem que causa a falha mais “barata” primeiro:

1. **Completude** — nenhuma célula pode estar vazia.
2. **Consistência com as pistas originais** — a resposta não pode alterar nenhum valor que já estava fixo no tabuleiro inicial.
3. **Ausência de conflitos** — reaproveita a mesma função `ValTabuleiro` usada na validação do tabuleiro inicial, evitando duplicar a lógica de checagem de linha/coluna/bloco.

7. Separação de leitura de arquivo por parâmetro

A função `lerTabuleiro` recebe o tabuleiro de destino como parâmetro (`int tab[9][9]`), em vez de escrever diretamente na variável global `tabuleiro`. Isso permite reutilizar a mesma função tanto para o tabuleiro original quanto para a resposta do usuário (`tabuleiroResposta`), sem sobrescrever um com o outro.

Validação do tamanho de cada linha: além de checar se cada caractere é um dígito válido (0–9), a leitura agora mede o tamanho real de cada linha do arquivo (com `strcspn`, ignorando quebras de linha) e rejeita qualquer linha que não tenha exatamente 9 caracteres. Sem essa checagem, um valor de dois dígitos (ex: 10) escrito por engano numa célula seria lido como dois caracteres separados (1 e 0), sendo silenciosamente interpretado como duas células distintas em vez de gerar um erro — a validação de tamanho fecha essa brecha, rejeitando a linha inteira nesse cenário.

8. Organização em subprogramas por responsabilidade única

O código foi dividido em seções e funções pequenas, cada uma com um propósito específico: manipulação da lista de dígitos, manipulação da lista de erros, cálculo de bloco, manipulação de candidatos, escolha de célula (MRV), resolução recursiva, leitura de arquivo e validação. Essa divisão evita funções extensas fazendo múltiplas tarefas, e torna cada parte do código testável e legível isoladamente.

9. Por que a solução é predominantemente imperativa

- O estado central (`tabuleiro[9][9]`) é mutado repetidamente ao longo da execução (leitura, tentativa, backtrack), em vez de cada etapa gerar uma nova cópia imutável do tabuleiro.
- O fluxo de controle é conduzido por estruturas clássicas do paradigma imperativo: `for`, `while`, `if/else`, `switch`, e recursão com efeito colateral (a recursão em `resolverSudoku` modifica o tabuleiro e os bitmasks diretamente a cada chamada, em vez de compor funções puras sem efeitos colaterais).
- As structs (`NoE`, `NoD`, `EstadoBitmask`) são registros de dados simples, com campos acessados diretamente (`->campo`), sem encapsulamento ou métodos vinculados — ou seja, modularização em C, e não uma abstração orientada a objetos disfarçada.